
DTU

02159 – Operating Systems

Week 1: OS Challenge

Xenofon (Fontas) Fafoutis
Professor
xefa@dtu.dk
www.compute.dtu.dk/~xefa

OS Challenge

- **Mandatory** group project for 02159
- The project has the form of a **game**

Learning Objectives

- Operating Systems (OS) manage the resources of a computer (cpu, memory, etc)
  - The resources are limited, thus must be managed efficiently
  - We need solutions that **work fast** (fastest team wins!)

- You will create a server application for **Linux** using the **C programming language**
  - You will learn how to program at the OS interface and use Linux system calls
  - You will use a deep understanding on how Linux works to develop the server
  - You will learn how to make experiments to compare and select approaches

Reverse Hashing Server

- Your goal is to make a server that handles **reverse hashing requests**
  - Hashing is a **deterministic** function that is **computationally hard to invert**
    - It is easy to get the hash from the input, but hard to get the input from the hash
  - Reverse hashing requires to **brute-force** all possible inputs until you get a match
    - Emulates a computationally-intensive task
- Your server will receive a hash value and answer back the input value that generates it
- Your server should be able to handle multiple requests at the same time
- Your server should be as fast as possible

Client – Request Generator

- You are provided with client application that generate requests

- The client has several configuration parameters that control:
  - The difficulty of the reverse hashing requests
  - The time between consecutive reverse hashing requests
  - The probability that a previous reverse hashing request is requested again
  - The priority level of the generated requests

Client-Server Protocol

- Your server must adhere 100% to a **specified protocol**
  - Not doing so means that it won't be able to communicate with the client!

- Client-Server Protocol
  - Based on TCP Sockets
  - Two packet formats: request and response packets
  - Byte order
    - Little-endian when processed on client/server
    - Big-endian (network order) when in transit

Benchmarks

- **Reliability:** Total correct answers over total requests
  – This is your first priority!

- **Score**: Weighted average request-response latency
  – Client measures the time between a request and a correct response (l)
  – This time is penalised proportionally to its priority level (p)

$$\texttt{score} = \frac{1}{\texttt{total}} \sum_{i=1}^{\texttt{total}} l_i \texttt{p}_i$$

Insights

- The task emulates the challenges a modern OS must face

- An OS handles requests from the users
  - Requests may arrive sporadically on in bursts
  - Requests often have a temporal locality
  - Requests have different priority levels

Group Formation

- You are free to set your own groups
- The recommended group size is 2-4 persons

- Start preparing from today! Do not wait for the group formation to be finalised
- Please, respect the deadlines

Git and DTU GitLab

- You must use a **git repository** for your project
- DTU's GitLab is the preferred platform for disseminating and submitting your project
- The structure of the repository must follow specific rules
  – I will automatically pull your source code, compile it, and execute it from a script
- The final submission should have a README.md file
  – Explain the structure of your repository
- Git logs will be used to detect free-riders

Process

- Milestone Deadline
  - You should submit a server that works (100% reliability)
- Final Deadline
  - You should submit a server that works fast (100% reliability + low score)
  - **Each team member must design and execute one experiment**
- Report
  - The groups should document the design of the final submitted solution
  - **Each team member must report one experiment**
  - If you have done multiple experiments, report only the best one

Experiment

- In the second stage, your goal is to design the fastest possible server
  - To do so, you should compare ideas and select the fastest to integrate in your server

- Experiment stages:
  - Based on theory, reading, background knowledge you create a hypothesis that makes sense (e.g. multiple threads is faster than multi processes)
  - You implement the new idea and you compare it with the existing one
    - The binary of a reference server implementation is also provided
  - The comparison should be conducted under the same experimental conditions
  - The conclusions derived by the experimental results drive the final decision

- The code of the experiments should be included in the repository (even if not used)
- The (best) experiment must be documented in the report

Assessment

- The group project is mandatory and counts towards the final grade
- However, the rankings are **just for fun**; your final grade is irrelevant to your team ranking
- Assessment Criteria
  - Assessment will be based on objective criteria such as correctness (100% reliability), code quality, documentation quality, quality of experiment

Important Dates and Deadlines

- Check OS Challenge Specifications document on DTU Learn

Getting Started

- Read carefully the OS Challenge project specification document
- Refresh your skills in C Programming, the GCC compiler, Makefile, and the Linux Command Line Interface
- Understand how git and GitLab works (you will likely have to use it via the terminal)
- Setup the testing environment and understand how the Vagrant virtualisation environment works
- Understand how to make a basic TCP Client/Server application on Linux
- Start today, don’t wait for group formation!

Tips and Advice

- You are free to manage your time, but **don't start at the last moment!**
  - The challenge can be difficult, especially if you are not experienced in C
- **Identify your missing background knowledge and start working on it**
  - How is my C programming? Am I comfortable with pointers?
  - Am I comfortable around the Linux Terminal?
  - Am I comfortable with Git? Vagrant?
- **Meet with your group mates regularly** (e.g. weekly)
  - Agree on collaboration rules on the first meeting (group contract)
  - Discuss your grade expectations and how much time you plan to put on the course
- **Make your hands dirty!**
  - Learning OS and OS programming requires practice

Questions?